

APPLICATION PROGRAMMING USING RTOS

Task synchronization using semaphores, Inter task communication: message queues and pipes, Remote procedure call- Timers and Interrupts-Memory management and I/O management

Task Management:

Task is the basic notion in RTOS.

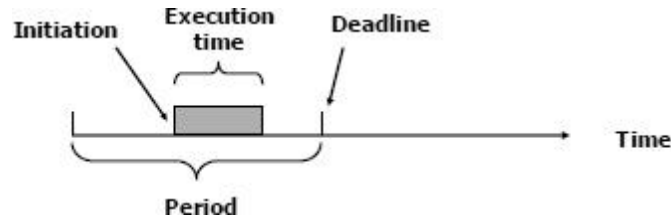


Fig:4.1:Typical RTOS Task Model

Periodic tasks: arriving at fixed frequency, can be characterized by 3 parameters (C,D,T) where, C = computing time, D = deadline and T = period. Also called Time-driven tasks, their activations are generated by timers.

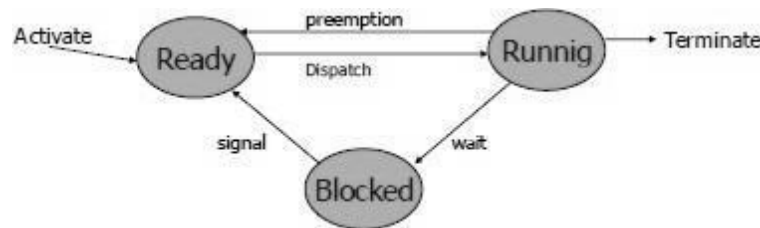


Fig:4.2:Task states

Non-Periodic or aperiodic tasks = all tasks that are not periodic, also known as Event driven, their activations may be generated by external interrupts. **Sporadic tasks**= aperiodic tasks with minimum inter arrival time T_{min} .

Managing tasks:

- Execution of quasi-parallel tasks on a processor using processes or threads by maintaining process states, process queuing, allowing for preemptive tasks and quick interrupt handling.
- CPU scheduling
- Process synchronization
- Inter-process communication
- Support of a real-time clock as an internal time reference

- **Task creation:** create a new TCB (task control block)
- **Task termination:** remove the TCB
- **Change Priority:** modify the TCB
- **State-inquiry:** read the TCB

Task management is depicted in the below diagram

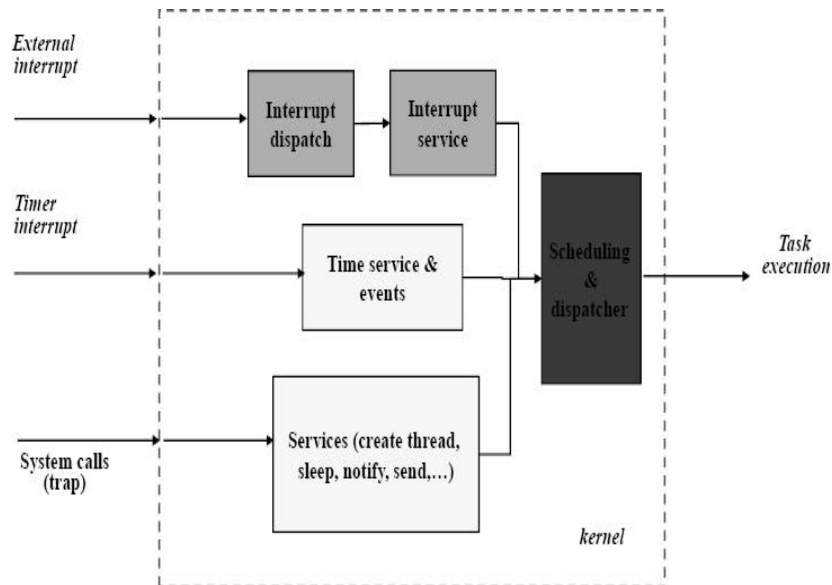


Fig:4.3: Block Diagram of Task management

Task synchronization:

In classical operating systems, synchronization and mutual exclusion is performed via semaphores and monitors. In real-time OS, special semaphores and a deep Integration into scheduling is necessary (priority inheritance protocols).

Further responsibilities: Initializations of internal data structures (tables, queues, task description blocks, semaphores)

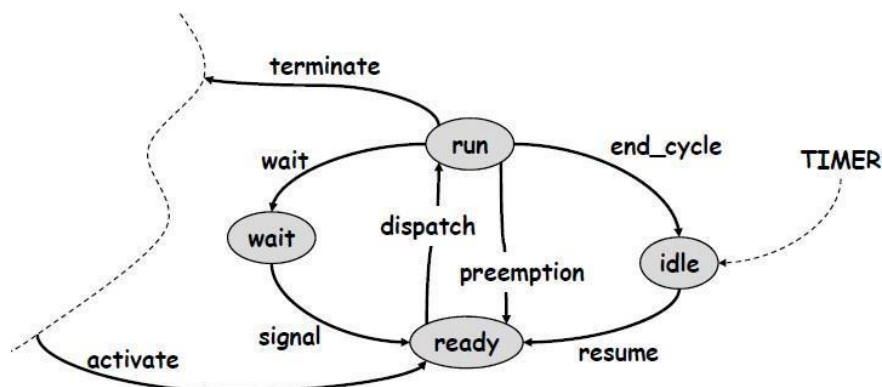


Fig:4.4: Minimal Set of Task States

Run:

A task enters this state as it starts executing on the processor

Ready:

State of those tasks that are ready to execute but cannot be executed, because the processor is assigned to another task.

Wait:

A task enters this state when it executes a synchronization primitive to wait for an event, e.g. a wait primitive on a semaphore. In this case, the task is inserted in a queue associated with the semaphore. The task at the head is resumed when the semaphore is unlocked by a signal primitive.

Idle:

A periodic job enters this state when it completes its execution and has to wait for the beginning of the next period.

Challenges for an RTOS:

- Creating an RT task, it has to get the memory without delay: this is difficult because memory has to be allocated and a lot of data structures, code segment must be copied/initialized
- The memory blocks for RT tasks must be locked in main memory to avoid access latencies due to swapping
- Changing run-time priorities dangerous: it may change the run-time behavior and predictability of the whole system

Inter task communication and Synchronization:

Inter process Communication (IPC) enables processes to communicate with each other to share information

- Pipes (half duplex)
- FIFOs(named pipes)
- Stream pipes (full duplex)
- Named stream pipes
- Message queues
- Semaphores
- Shared Memory
- Sockets
- Streams
- Pipes

Oldest (and perhaps simplest) form of UNIX IPC

- Half duplex
- Data flows in only one direction
- Only usable between processes with a common ancestor
- Usually parent-child
- Also child-child

In computing, a named pipe (also known as a FIFO) is one of the methods for intern-process communication.

- It is an extension to the traditional pipe concept on Unix. A traditional pipe is “unnamed” and lasts only as long as the process.
- A named pipe, however, can last as long as the system is up, beyond the life of the process. It can be deleted if no longer used.
- Usually a named pipe appears as a file and generally processes attach to it for inter-process communication. A FIFO file is a special kind of file on the local storage which allows two or more processes to communicate with each other by reading/writing to/from this file.
- A FIFO special file is entered into the filesystem by calling *mkfifo()* in C. Once we have created a FIFO special file in this way, any process can open it for reading or writing, in the same way as an ordinary file. However, it has to be open at both ends simultaneously before you can proceed to do any input or output operations on it.

Understanding Pipes

- Within a process
- Writes to files can be read on files
- Not very useful
- Between processes
- After a fork()
- Writes to files by one process can be read on files by the other

Using Pipes:

- Usually, the unused end of the pipe is closed by the process
- If process A is writing and process B is reading, then process A would close files[0] and process B would close files[1]
- Reading from a pipe whose write end has been closed returns 0 (end of file)
- Writing to a pipe whose read end has been closed generates SIGPIPE
- PIPE_BUF specifies kernel pipe buffer size

Creating a Pipe

The primitive for creating a pipe is the pipe function. This creates both the reading and writing ends of the pipe. It is not very useful for a single process to use a pipe to talk to itself. In typical use, a process creates a pipe just before it forks one or more child processes. The pipe is then used for communication either between the parent or child processes, or between two sibling processes.

The pipe function is declared in the header file unistd.h. Here is an example of a simple program that creates a pipe. The parent process writes data to the pipe, which is read by the child process.

```

#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

/* Read characters from the pipe and echo them to stdout. */

void
read_from_pipe (int file)
{
    FILE *stream;
    int c;
    stream = fdopen (file, "r");
    while ((c = fgetc (stream)) != EOF)
        putchar (c);
    fclose (stream);
}

/* Write some random text to the pipe. */

void
write_to_pipe (int file)
{
    FILE *stream;
    stream = fdopen (file, "w");
    fprintf (stream, "hello, world!\n");
    fprintf (stream, "goodbye, world!\n");
    fclose (stream);
}

int
main (void)
{
    pid_t pid;
    int mypipe[2];

    /* Create the pipe. */
    if (pipe (mypipe))
    {
        fprintf (stderr, "Pipe failed.\n");
        return EXIT_FAILURE;
    }
}

```

```

/* Create the child process. */
pid = fork ();
if (pid == (pid_t) 0)
{
    /* This is the child process.
       Close other end first. */
    close (mypipe[1]);
    read_from_pipe (mypipe[0]);
    return EXIT_SUCCESS;
}
else if (pid < (pid_t) 0)
{
    /* The fork failed. */
    fprintf (stderr, "Fork failed.\n");
    return EXIT_FAILURE;
}
else
{
    /* This is the parent process.
       Close other end first. */
    close (mypipe[0]);
    write_to_pipe (mypipe[1]);
    return EXIT_SUCCESS;
}
}

```

Using Pipes for synchronization and communication:

- Once you have a pipe or pair of pipes set up, you can use it/them to Signal events (one pipe)
- Wait for a message
- Synchronize (one or two pipes)
- Wait for a message or set of messages
- You send me a message when you are ready, then I'll send you a message when I am ready
- Communicate (one or two pipes)
- Send messages back and forth
- `popen()` and `pclose()`:

FIFO

A FIFO special file is similar to a pipe, except that it is created in a different way. Instead of being an anonymous communications channel, a FIFO special file is entered into the file system by calling `mkfifo`.

Once you have created a FIFO special file in this way, any process can open it for reading or writing, in the same way as an ordinary file. However, it has to be open at both ends simultaneously before you can proceed to do any input or output operations on it. Opening a FIFO for reading normally blocks until some other process opens the same FIFO for writing, and vice versa.

First: Co processes—Nothing more than a process whose input and output are both redirected from another process

FIFOs—named pipes

With regular pipes, only processes with a common ancestor can communicate With FIFOs, any two processes can communicate

Creating and opening a FIFO is just like creating and opening a file

FIFO details:

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```
intmkfifo(constchar *pathname, mode_tmode);
```

The modeargument is just like in open()

Can be opened just like a file

When opened, O_NONBLOCK bit is important

Not specified: open() for reading blocks until the FIFO is opened by a writer

Specified: open() returns immediately, but returns an error if opened for writing and no reader exists

Four Queue States:

- 1. Both the sending and receiving queue are running.**
- 2. Only the sender is executing.**
- 3. Sender not executing, but receiver is running.**
- 4. Neither the sender nor the receiver are running.**

Parts of Message Queue:

The queue that is local to the sending machine is called the source queue. The local queue of the

receiver is called the destination queue. Queues are managed by queue managers. Mapping of queues to network locations.

Message sending methods:

- 1. Message Queues**
- 2. Mail box:**

Mail box:

Mailboxes are similar to queues.

- **Capacity of mailboxes is usually fixed after initialization**
- **Some RTOS allow only a single message in the mailbox. (mailbox is either full or empty)**
- **Some RTOS allow prioritization of messages**

Mailbox functions at OS

Some OS provide the mailbox and queue both IPC functions. When the IPC functions for mailbox are not provided by an OS, then the OS employs queue for the same purpose. A mailbox of a task can receive from other tasks and has a distinct ID. Mailbox (for message) is an IPC through a message at an OS that can be received only one single destined task for the message from the tasks. Two or more tasks cannot take message from same Mailbox. A task on an OS function call puts (means post and also send) into the mailbox only a pointer to a mailbox message. Mailbox message may also include a header to identify the message-type specification.

OS provides for inserting and deleting message into the mailbox message pointer. Deleting means message-pointer pointing to Null. Each mailbox for a message need initialization (creation) before using the functions in the scheduler for the message queue and message pointer pointing to Null. There may be a provision for multiple mailboxes for the multiple types or destinations of messages. Each mailbox has an ID. Each mailbox usually has one message pointer only, which can point to message.

Mailbox Types

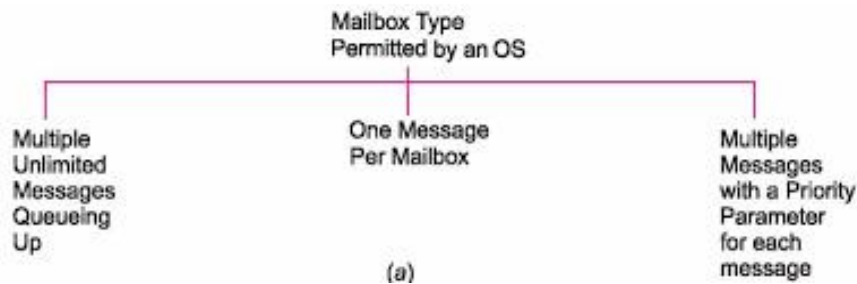


Fig:4.5:Classification of mailbox

Mailbox IPC features

When an OS call is to post into the mailbox, the message bytes are as per the pointed number of bytes by the mailbox message pointer.

Mailbox Related Functions at the OS

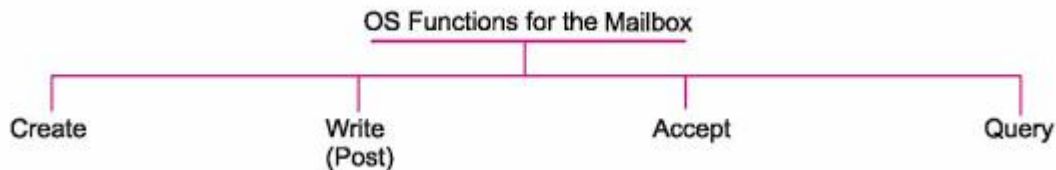


Fig:4.6:Features of mailbox

Mailbox IPC functions

1. **OSMBoxCreate** creates a box and initializes the mailbox contents with a NULL pointer at *msg . 2. **OSMBoxPost** sends at *msg, which now does not point to Null.
2. An ISR can also post into mailbox for a task
3. **OSMBoxWait (Pend)** waits for *msg not Null, which is read when not Null and again *msg points to Null. • The time out and error handling function can be provided with Pend function argument. • ISR not permitted to wait for message into mailbox. Only the task can wait
4. **OSMBoxAccept** reads the message at *msg after checking the presence yes or no [No wait.] Deletes (reads) the mailbox message when read and *msg again points to Null
• An ISR can also accept mailbox message for a task
5. **OSMBoxQuery** queries the mailbox *msg.
6. **OSMBoxDelete**

Event Management and Memory Management

Tasks are event driven or time driven. In RTOS environment there is less significance for event driven mechanism. Just as processes share the CPU, they also share physical memory. The concept of a logical address space that is bound to a separate physical address space is central to proper memory management.

- Logical address – generated by the CPU; also referred to as virtual address
- Physical address – address seen by the memory unit

Logical and physical addresses are the same in compile-time and load time address binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme. Re locatable Means that the program image can reside anywhere in physical memory. Binding Programs need real memory in which to reside. When is the location of that real memory determined. This is called mapping logical to physical addresses. This binding can be done at compile/link time. Converts symbolic to re locatable. Data used within compiled source is offset within object module.

Compiler: If it's known where the program will reside, then absolute code is generated. Otherwise compiler produces re locatable code.

Load: Binds re locatable to physical. Can find best physical location.

Execution: The code can be moved around during execution. Means flexible virtual Mapping

Example of Memory Usage:

Calculation of an effective address

Fetch from instruction

Use index offset

Example: (Here index is a pointer to an address)

loop:

load register, index

add 42, register store

register, index inc

index

skip_equal index, final_address

branch loop

This binding can be done at compile/link time. Converts symbolic to re locatable. Data used within compiled source is offset within object module. Can be done at load time. Binds reloadable to physical. Can be done at run time. Implies that the code can be moved around during execution.

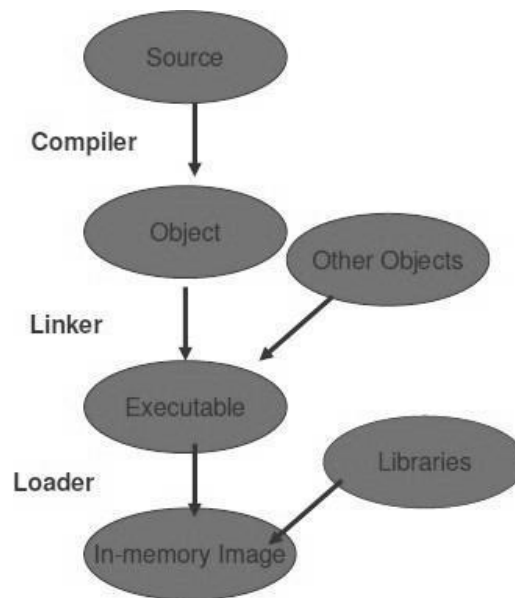


Fig:4.7:Design of software development tools

Message queues

A message queue is an object used for inter task communication through which task send or receive messages placed in a shared memory. The queue may follow

1) First In First Out (FIFO), 2) Last in First Out(LIFO) or 3) Priority (PRI) sequence. Usually, a message queue comprises of an associated queue control block (QCB), name, unique ID, memory buffers, queue length, maximum message length and one or more task waiting lists. A message queue with a length of 1 is commonly known as a mailbox.

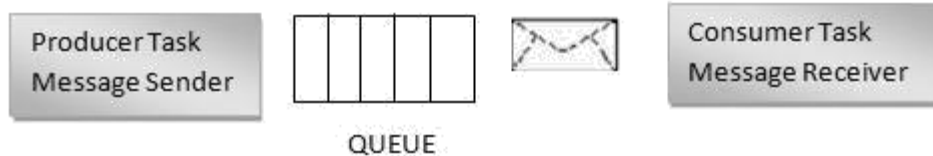


Fig:4.8: Message queues

Pipes

A pipe is an object that provide simple communication channel used for unstructured data exchange among tasks. A pipe does not store multiple messages but stream of bytes. Also, data flow from a pipe cannot be prioritized.

Remote procedure call (RPC)

Remote Procedure Call (RPC) is a powerful technique for constructing distributed, client-server based applications. It is based on extending the conventional local procedure calling so that the called procedure need not exist in the same address space as the calling procedure. A remote procedure call is an inter process communication technique that is used for client-server based applications. It is also known as a subroutine call or a function call. A client has a request message that the RPC translates and sends to the server. This request may be a procedure or a function call to a remote server. When the server receives the request, it sends the required response back to the client. The client is blocked while the server is processing the call and only resumed execution after the server is finished.

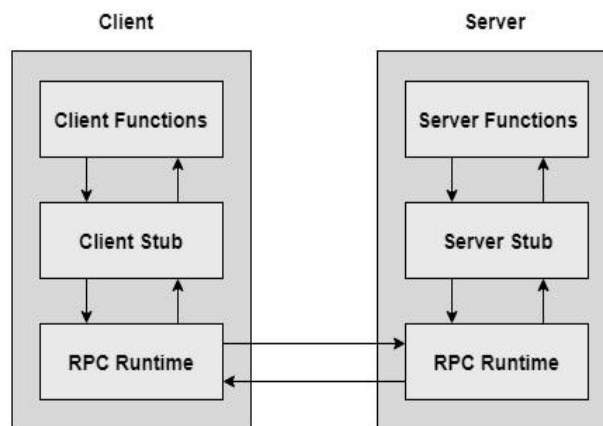


Fig:4.9: Remote procedure call

The sequence of events in a remote procedure call are given as follows :

1. A client invokes a client stub procedure, passing parameters in the usual way. The client stub resides within the client's own address space.

2. The client stub marshalls (pack) the parameters into a message. Marshalling includes converting the representation of the parameters into a standard format, and copying each parameter into the message.
3. The client stub passes the message to the transport layer, which sends it to the remote server machine.
4. On the server, the transport layer passes the message to a server stub, which demarshalls(unpack) the parameters and calls the desired server routine using the regular procedure call mechanism.
5. When the server procedure completes, it returns to the server stub (e.g., via a normal procedure call return), which marshalls the return values into a message. The server stub then hands the message to the transport layer.
6. The transport layer sends the result message back to the client transport layer, which hands the message back to the client stub.
7. The client stub demarshalls the return parameters and execution returns to the caller.

RPC Issues

1. **RPC Runtime:** RPC run-time system is a library of routines and a set of services that handle the network communications that underlie the RPC mechanism. In the course of an RPC call, client-side and server-side run-time systems' code handle binding, establish communications over an appropriate protocol, pass call data between the client and server, and handle communications errors.
2. **Stub:** The function of the stub is to provide transparency to the programmer-written application code. On the client side, the stub handles the interface between the client's local procedure call and the run-time system, marshaling and unmarshaling data, invoking the RPC run-time protocol, and if requested, carrying out some of the binding steps. On the server side, the stub provides a similar interface between the run-time system and the local manager procedures that are executed by the server.
3. **Binding:** The most flexible solution is to use dynamic binding and find the server at run time when the RPC is first made. The first time the client stub is invoked, it contacts a name server to determine the transport address at which the server resides.

Advantages of Remote Procedure Call

Some of the advantages of RPC are as follows

- Remote procedure calls support process oriented and thread oriented models.
- The internal message passing mechanism of RPC is hidden from the user.
- The effort to re-write and re-develop the code is minimum in remote procedure calls.
- Remote procedure calls can be used in distributed environment as well as the local environment.
- Many of the protocol layers are omitted by RPC to improve performance.

Disadvantages of Remote Procedure Call

Some of the disadvantages of RPC are as follows

- The remote procedure call is a concept that can be implemented in different ways. It is not a standard.
- There is no flexibility in RPC for hardware architecture. It is only interaction based.
- There is an increase in costs because of remote procedure call

Memory Management

Memory management is one of the most important subsystems of any operating system for computer control systems, and is even more critical in a RTOS than in standard operating systems. Firstly, the speed of memory allocation is important in a RTOS. A standard memory allocation scheme scans a linked list of indeterminate length to find a free memory block; however, memory allocation has to occur in a fixed time in a RTOS. Secondly, memory can become fragmented as free regions become separated by regions that are in use, causing a program to stall, unable to get memory, even though there is theoretically enough available. Memory allocation algorithms that slowly accumulate fragmentation may work perfectly well for desktop machines rebooted every day or so but are unacceptable for embedded systems that often run for months without rebooting.

Memory management is the process by which a computer control system allocates a limited amount of physical memory among its various processes (or tasks) in a way that optimizes performance. Actually, each process has its own private address space, initially divided into three logical segments: text, data, and stack. The text segment is read-only and contains the machine instructions of a program, the data and stack segments are both readable and writable. The data segment contains the initialized and non-initialized data portions of a program, whereas the stack segment holds the application's run-time stack. On most machines, this is extended automatically by the kernel as the process executes. This is done by making a system call, but change to the size of a text segment only happens when its contents are overlaid with data from the file system, or when debugging takes place. The initial contents of the segments of a child process are duplicates of the segments of its parent.

The contents of a process address space do not need to be completely in place for a process to execute. If a process references a part of its address space that is not resident in main memory, the system pages the necessary information into memory. When system resources are scarce, the system uses a two-level approach to maintain available resources. If a modest amount of memory is available, the system will take memory resources away from processes if these resources have not been used recently. Should there be a severe resource shortage, the system will resort to swapping the entire context of a process to secondary storage. This paging and swapping done by the system are effectively transparent to processes, but a process may advise the system about expected future memory utilization as a performance aid. A common technique for doing the above is virtual memory, which simulates a much larger address space than is actually available, using a reserved disk area for objects that are not in physical memory. The operating system's kernel often performs memory allocations that are needed for only the duration of a single system call. In a user

process, such short-term memory would be allocated on the run-time stack. Because the kernel has a limited run-time stack, it is not feasible to allocate even moderately- sized blocks of memory on it, so a more dynamic mechanism is needed. For example, when the system must translate a path name, it must allocate a 1-kbyte buffer to hold the name. Other blocks of memory must be more persistent than a single system call, and thus could not be allocated on the stack even if there was space. An example is protocol-control blocks that remain throughout the duration of a network connection.

This section discusses virtual memory techniques, memory allocation and deallocation, memory protection and memory access control.

Virtual memory

When it is executing a program, the microprocessor reads an instruction from memory and decodes it. At this point it may need to fetch or store the contents of a location in memory, so it executes the instruction and then moves on to the next. In this way the microprocessor is always accessing memory, either to fetch instructions or to fetch and store data. In a virtual memory system all of these addresses are virtual, and not physical addresses. They are converted into physical addresses by the microprocessor, based on information held in a set of tables maintained by the operating system.

The operating system uses virtual memory to manage the memory requirements of its processes by combining physical memory with secondary memory (swap space) on a disk, usually located on a hardware disk drive. Diskless systems use a page server to maintain their swap areas on the local disk (extended memory). The translation from virtual to physical addresses is implemented by a memory management unit (MMU), which may be either a module of the CPU, or an auxiliary, closely coupled chip. The operating system is responsible for deciding which parts of the program's simulated main memory are kept in physical memory, and also maintains the translation tables that map between virtual and physical addresses. Three techniques of implementing virtual memory; paging, swapping and segmentation.

(1) Paging

Almost all implementations of virtual memory divide the virtual address space of an application program into pages; a page is a block of contiguous virtual memory addresses. Here, the low-order bits of the binary representation of the virtual address are preserved, and used directly as the low-order bits of the actual physical address; the high-order bits are treated as a key to one or more address translation

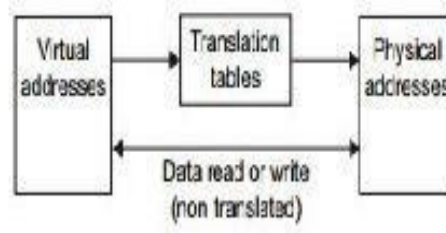


Fig: 4 .10 :Address translation between physical and virtual memory

tables, which provide the high-order bits with the actual physical address. For this reason, a range of consecutive addresses in the virtual address space, whose size is a power of two, will be translated to a corresponding range of consecutive physical addresses. The memory referenced by such a range is called a page. The page size is typically in the range 512 8192 bytes (with 4 kB currently being very common), though 4 MB or even larger may be used for special purposes. (Using the same or a related mechanism, contiguous regions of virtual memory larger than a page are often mappable to contiguous physical memory for purposes other than virtualization, such as setting access and caching control bits.)

Almost all implementations use page tables to translate the virtual addresses seen by the application into physical addresses (also referred to as real addresses) used by the hardware. The operating system stores the address translation tables, i.e. the mappings from virtual to physical page numbers, in a data structure known as a page table. When the CPU tries to reference a memory location that is marked as unavailable, the MMU responds by raising an exception (commonly called a page fault) with the CPU, which then jumps to a routine in the operating system. If the page is in the swap area, this routine invokes an operation called a page swap, to bring in the required page.

The operating systems can have one page table or a separate page table for each application. If there is only one, different applications running at the same time will share a single virtual address space, i.e. they use different parts of a single range of virtual addresses. The operating systems which use multiple page tables provide multiple virtual address spaces, so concurrent applications seem to use the same range of virtual addresses, but their separate page tables redirect to different real addresses

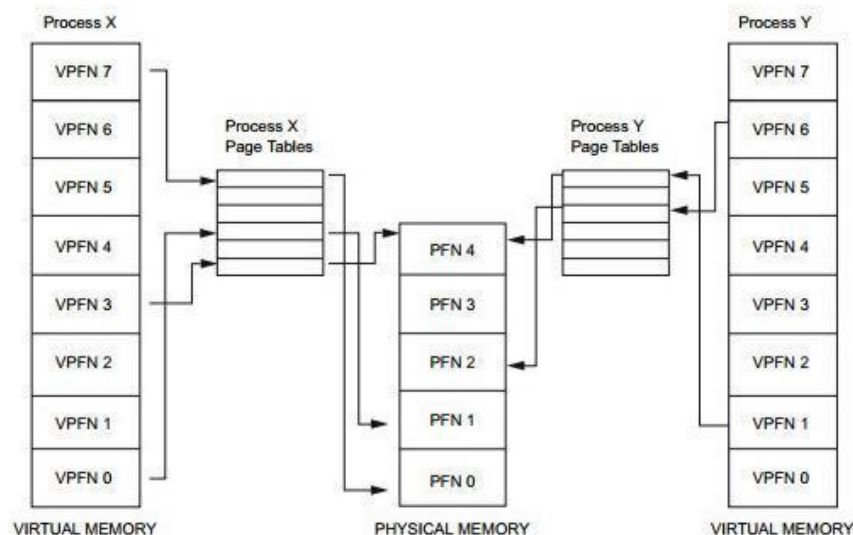


Fig:4.11: Abstract model for mapping virtual addresses to physical addresses in the implementation of virtual memory

The above figure shows the virtual address spaces of two processes, X and Y, each with their own page tables, which map each process's virtual pages into physical pages in memory. This shows that process X's virtual page frame number 0 is mapped into memory at physical page frame number 1 and that process Y's virtual page frame number 1 is mapped into physical page frame number 4. Each entry in the theoretical page table contains the following information: (1) a valid flag that indicates whether this page table entry is valid; (2) the physical page frame number that this entry describes; (3) the access control information that describes how the page may be used.

(2) Swapping

Swap space is a portion of hard disk used for virtual memory that is usually a dedicated partition (i.e., a logically independent section of a hard disk drive), created during the installation of the operating system. Such a partition is also referred to as a swap partition. However, swap space can also be a special file. Although it is generally preferable to use a swap partition rather than a file, sometimes it is not practical to add or expand a partition when the amount of RAM is being increased. In such case, a new swap file can be created with a system call to mark a swap space.

It is also possible for a virtual page to be marked as unavailable because the page was never previously allocated. In such cases, a page of physical memory is allocated and filled with zeros, the page table is modified to describe it, and the program is restarted as above

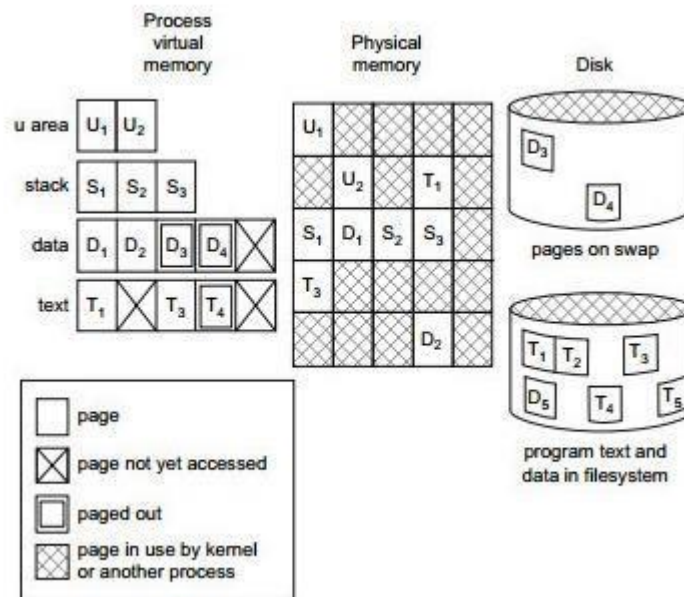


Fig:4.12: Page swapping for implementing virtual memory

The above figure illustrates how the virtual memory of a process might correspond to what exists in physical memory, on swap, and in the file system. The U-area of a process consists of two 4 kB pages (displayed here as U₁ and U₁) of virtual memory containing information about the process that is needed by the system during execution. In this example, these pages are shown in physical memory, and the data pages, D₃ and D₄, are shown as being paged out to the swap area on disk. The text page, T₄, has also been paged out, but it is

not written to the swap area as it exists in the file system. Those pages that have not yet been accessed by the process (D5, T2, and T5) do not occupy any resources in physical memory or in the swap area. The page swap operation involves a series of steps. Firstly it selects a page in memory; for example, a page that has not been recently accessed and (preferably) has not been modified since it was last read. If the page has been modified, the process writes the modified page to the swap area. The next step in the process is to read in the information in the needed page (the page corresponding to the virtual address the original program was trying to reference when the exception occurred) from the swap file. When the page has been read in, the tables for translating virtual addresses to physical addresses are updated to reflect the revised contents of physical memory. Once the page swap completes, it exits, the program is restarted and returns to the point that caused the exception.

(3) Segmentation

Some operating systems do not use paging to implement virtual memory, but use segmentation instead. For an application process, segmentation divides its virtual address space into variable-length segments, so a virtual address consists of a segment number and an offset within the segment. Memory is always physically addressed with a single number (called absolute or linear address). To obtain it, the microprocessor looks up the segment number in a table to find a segment descriptor. This contains a flag indicating whether the segment is present in main memory and, if so, the address of its starting point (segment's base address) and its length. It checks whether the offset within the segment is less than the length of the segment and, if not, generates an interrupt. If a segment is not present in main memory, a hardware interrupt is raised to the operating system, which may try to read the segment into main memory, or to swap it in. The operating system may need to remove other segments (swap out) in order to make space for the segment to be read in.

The difference between virtual memory implementations that use pages and those using segments is not only about the memory division. Sometimes the segmentation is actually visible to the user processes, as part of the semantics of the memory model. In other words, instead of a process just having a memory which looks like a single large vector of bytes or words, it is more structured. This is different from using pages, which does not change the model visible to the process. This has important consequences. It is possible to combine segmentation and paging, usually by dividing each segment into pages. In such systems, virtual memory is usually implemented by paging, with segmentation used to provide memory protection. The segments reside in a 32-bit linear paged address space, which segments can be moved into and out of, and pages in that linear address space can be moved in and out of main memory, providing two levels of virtual memory. This is quite rare, however, most systems only use paging.

Memory allocation and deallocation

The inefficient allocation or deallocation of memory can be detrimental to system performance. The presence of wasted memory in a block is called internal fragmentation, and it occurs because the size that was requested was smaller than that allocated. The result is a block of unusable memory, which is considered as allocated when not being used. The reverse situation is called external fragmentation, when blocks of memory are freed, leaving non-

contiguous holes. If these holes are not large, they may not be usable because further requests for memory may call for larger blocks. Both internal and external fragmentation results in unusable memory. Memory allocation and deallocation is a process that has several layers of application. If one application fails, another operates to attempt to resolve the request. This whole process is called dynamic memory management. Memory allocation is controlled by a subsystem called malloc, which controls the heap, a region of memory to which memory allocation and deallocation occurs. The reallocation of memory is also under the control of malloc. In malloc, the allocation of memory is performed by two subroutines, malloc and calloc. Deallocation is performed by the free subroutine, and reallocation is performed by the subroutine known as realloc. In deallocation, those memory blocks that have been deallocated are returned to the binary tree at its base. Thus, a binary tree can be envisioned as a sort of river of information, with deallocated memory flowing in at the base and allocated memory flowing out from the tips.

Garbage collection is another term associated with deallocation of memory. This refers to an automated process that determines what memory is no longer in use, and so recycles it. The automation of garbage collection relieves the user of time-consuming and error-prone tasks. There are a number of algorithms for the garbage collection process, all of which operate independently of malloc.

Memory allocation and deallocation can be categorized as static or dynamic.

(1) Static memory allocation

Static memory allocation refers to the process of allocating memory at compile-time, before execution. One way to use this technique involves a program module (e.g., function or subroutine) declaring static data locally, such that these data are inaccessible to other modules unless references to them are passed as parameters or returned. A single copy of this static data is retained and is accessible through many calls to the function in which it is declared. Static memory allocation therefore has the advantage of modularizing data within a program so that it can be used throughout run-time. The use of static variables within a class in object-oriented programming creates a single copy of such data to be shared among all the objects of that class.

(2) Dynamic memory allocation

Dynamic memory allocation is the allocation of memory storage for use during the run-time of a program, and is a way of distributing ownership of limited memory resources among many pieces of data and code. A dynamically allocated object remains allocated until it is deallocated explicitly, either by the programmer or by a garbage collector; this is notably different from automatic and static memory allocation. It is said that such an object has dynamic lifetime. Memory pools allow dynamic memory allocation comparable to malloc, or the operator “new”. As those implementations suffer from fragmentation because of variable block sizes, it can be impossible to use them in a real-time system due to performance problems.

A more efficient solution is to pre-allocate a number of memory blocks of the same size, called the memory pool. The application can allocate, access, and free blocks represented by handles at run- time. Fulfilling an allocation request, which involves finding a block of unused memory of a certain size in the heap, is a difficult problem. A wide variety of solutions have been proposed, and some of the most commonly used are discussed here.

(a) Free lists

A free list is a data structure used in a scheme for dynamic memory allocation that operates by connecting unallocated regions of memory together in a linked list, using the first word of each unallocated region as a pointer to the next. It is most suitable for allocating from a memory pool, where all objects have the same size. Free lists make the allocation and deallocation operations very simple. To free a region, it is just added to the free list. To allocate a region, we simply remove a single region from the end of the free list and use it. If the regions are variable-sized, we may have to search for a large enough region, which can be expensive. Free lists have the disadvantage, inherited from linked lists, of poor locality of reference and thus poor data cache utilization, and they provide no way of consolidating adjacent regions to fulfill allocation requests for large regions. Nevertheless, they are still useful in a variety of simple applications where a full-blown memory allocator is unnecessary, or requires too much overhead.

(b) Paging

As mentioned earlier, the memory access part of paging is done at the hardware level through page tables, and is handled by the MMU. Physical memory is divided into small blocks called pages (typically 4 kB or less in size), and each block is assigned a page number. The operating system may keep a list of free pages in its memory, or may choose to probe the memory each time a request is made (though most modern operating systems do the former). In either case, when a program makes a request for memory, the operating system allocates a number of pages to it, and keeps a list of allocated pages for that particular program in memory.

Memory protection

The topic of memory management in this section addresses a different set of constructs related to physical and virtual memory: protected memory, infinite amount of memory, and transparent sharing. Perhaps the simplest model for using memory is to provide single programming without memory protection, where each process (or task) runs with a range of physical memory addresses. Given that a single-programming environment allows only one process to run at a time, this can use the same physical addresses every time, even across reboots. Typically, processes use the lower memory addresses (low memory), and an operating system uses the higher memory addresses (high memory). An application process can address any physical memory location.

One step beyond the single-programming model is to provide multiprogramming without memory protection. When a program is copied into memory, a linker-loader alters the code of the program (loads, stores, jumps) to use the address of where the program lands in memory. In this environment, bugs in any program can cause other programs to crash, even the operating system. The third model is to have a multitasking operating system with memory protection, which keeps user programs from crashing one another and the operating system. Typically, this is achieved by two hardware-supported mechanisms: address translation and dual-mode operation.

(1) Address translation

Each process is associated with an address space, or all the physical addresses it can touch. However, each process appears to own the entire memory, with the starting virtual address of 0. The missing piece is a translation table that translates every memory reference from virtual addresses to physical addresses. Translation provides protection because there is no way for a process to talk about other processes' address, and it has no way of touching the code or data of the operating system. The operating system uses physical addresses directly, and involves no translation. When an exception occurs, the operating system is responsible for allocating an area of physical memory to hold the missing information (and possibly in the process pushing something else out to disk), bringing the relevant information in from the disk, updating the translation tables, and finally resuming execution of the software that incurred the exception.

(2) Dual-mode operation

Translation tables can offer protection only if a process cannot alter their content. Therefore, a user process is restricted to only touching its address space under the user mode. A CPU can change from kernel to user mode when starting a program, or vice versa through either voluntary or involuntary mechanisms. The voluntary mechanism uses system calls, where a user application asks the operating system to do something on its behalf. A system call passes arguments to an operating system, either through registers or copying from the user memory to the kernel memory. A CPU can also be switched from user to kernel mode involuntarily by hardware interrupts (e.g., I/O) and program exceptions (e.g., segmentation fault).

On system calls, interrupts, or exceptions, hardware atomically performs the following steps:

(1) sets the CPU to kernel mode; (2) saves the current program counter; (3) jumps to the handler in the kernel (the handler saves old register values).

Unlike threads, context switching among processes also involves saving and restoring pointers to translation tables. To resume execution, the kernel reloads old register values, sets the CPU to user mode, and jumps to the old program counter. Communication among address spaces is required in this operation. Since address spaces do not share memory, processes have to perform inter-process communication (IPC) through the kernel, which can allow bugs to propagate from one program to another. Protection by hardware can be prohibitively slow, since applications have to be structured to run in separate address spaces to achieve fault isolation. In the case of complex applications built by multiple vendors, it

may be desirable for two different programs to run in the same address space, with guarantees that they cannot trash each other's code or data. Strong typing and software fault isolation are used to ensure this.

(a) Protection via strong typing

If a programming language disallows the misuse of data structures, a program may trash another, even in the same address space. With some object-oriented programming, programs can be downloaded over the net and run safely because the language, compiler, and run-time system prevents the program from doing bad things (e.g., make system calls). For example, Java defines a separate virtual machine layer, so a Java program can run on different hardware and operating systems. The downside of this protection mechanism is the requirement to learn a new language.

(b) Protection via software fault isolation

A language-independent approach is to have compilers generate code that is proven safe (e.g., a pointer cannot reference illegal addresses). For example, a pointer can be checked before it is used in some applications.

Memory access control

Dealing with race conditions is also one of the difficult aspects of memory management. To manage memory access requests coming from the system, a scheduler is necessary in the application layer or in the kernel, in addition to the MMU as a hardware manager. The most common way of protecting data from concurrent access by the memory access request scheduler is memory request contention. The semantics and methodologies of memory access request contention should be the same as for I/O request contention.

Timer Management

Tasks need to be performed after scheduled durations. To keep track of the delays, timers- relative and absolute are provided in RTOS.

Interrupt and event handling

RTOS provides various functions for interrupt and event handling, viz., Defining interrupt handler, creation and deletion of ISR, referencing the state of an ISR, enabling and disabling of an interrupt, etc. It also restricts interrupts from occurring when modifying a data structure, minimize interrupt latencies due to disabling of interrupts when RTOS is performing critical operations, minimizes interrupt response times.

Device I/O Management

One of the important jobs of an Operating System is to manage various I/O devices including mouse, keyboards, touch pad, disk drives, display adapters, USB devices, Bit-mapped screen, LED, Analog-to-digital converter, On/off switch, network connections, audio I/O, printers etc. An I/O system is required to take an application I/O request and send it to the physical device, then take whatever response comes back from the device and send it to the application. I/O devices can be divided into two categories –

- **Block devices** – A block device is one with which the driver communicates by sending entire blocks of data. For example, Hard disks, USB cameras, Disk-On-Key etc.
- **Character devices** – A character device is one with which the driver communicates by sending and receiving single characters (bytes, octets). For example, serial ports, parallel ports, sounds cards etc

Device Controllers

Device drivers are software modules that can be plugged into an OS to handle a particular device. Operating System takes help from device drivers to handle all I/O devices. The Device Controller works like an interface between a device and a device driver. I/O units (Keyboard, mouse, printer, etc.) typically consist of a mechanical component and an electronic component where electronic component is called the device controller. There is always a device controller and a device driver for each device to communicate with the Operating Systems. A device controller may be able to handle multiple devices. As an interface its main task is to convert serial bit stream to block of bytes, perform error correction as necessary.

Any device connected to the computer is connected by a plug and socket, and the socket is connected to a device controller. Following is a model for connecting the CPU, memory, controllers, and I/O devices where CPU and device controllers all use a common bus for communication.

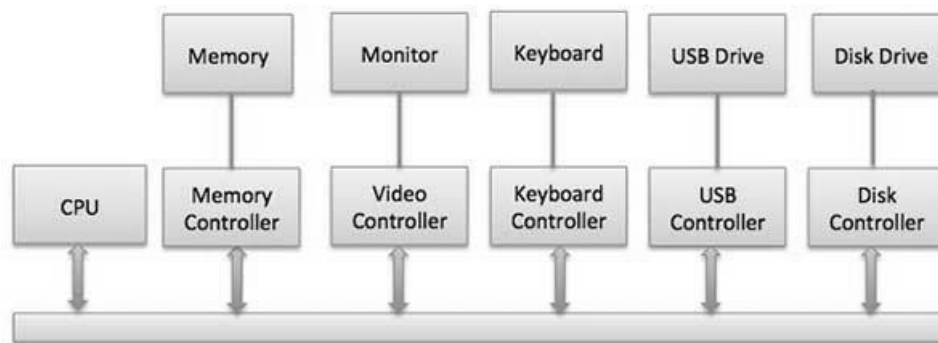


Fig:4.13: Model for connecting the CPU to peripherals

- **Synchronous I/O** – In this scheme CPU execution waits while I/O proceeds
- **Asynchronous I/O** – I/O proceeds concurrently with CPU execution

Communication to I/O Devices

The CPU must have a way to pass information to and from an I/O device. There are three approaches available to communicate with the CPU and Device.

- Special Instruction I/O
- Memory-mapped I/O
- Direct memory access (DMA)

Special Instruction I/O

This uses CPU instructions that are specifically made for controlling I/O devices. These instructions typically allow data to be sent to an I/O device or read from an I/O device.

Memory-mapped I/O

When using memory-mapped I/O, the same address space is shared by memory and I/O devices. The device is connected directly to certain main memory locations so that I/O device can transfer block of data to/from memory without going through CPU.

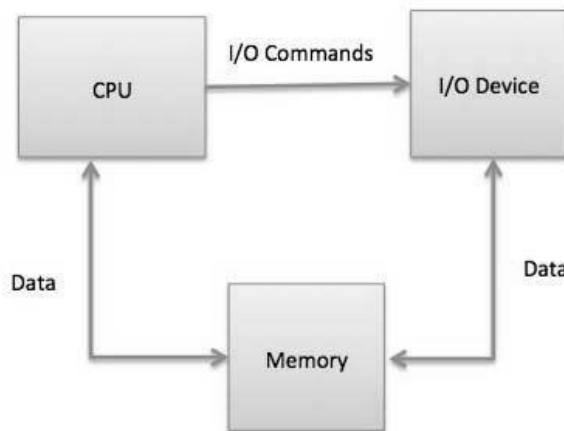


Fig:4.14: Memory-mapped I/O

While using memory mapped IO, OS allocates buffer in memory and informs I/O device to use that buffer to send data to the CPU. I/O device operates asynchronously with CPU, interrupts CPU when finished. The advantage to this method is that every instruction which can access memory can be used to manipulate an I/O device. Memory mapped IO is used for most high-speed I/O devices like disks, communication interfaces.

Direct Memory Access (DMA)

Slow devices like keyboards will generate an interrupt to the main CPU after each byte is transferred. If a fast device such as a disk generated an interrupt for each byte, the operating system would spend most of its time handling these interrupts. So a typical computer uses direct memory access (DMA) hardware to reduce this overhead. Direct Memory Access (DMA) means CPU grants I/O module authority to read from or write to memory without involvement. DMA module itself controls exchange of data between main

memory and the I/O device. CPU is only involved at the beginning and end of the transfer and interrupted only after entire block has been transferred. Direct Memory Access needs a special hardware called DMA controller (DMAC) that manages the data transfers and arbitrates access to the system bus. The controllers are programmed with source and destination pointers (where to read/write the data), counters to track the number of transferred bytes, and settings, which includes I/O and memory types, interrupts and states for the CPU cycles.

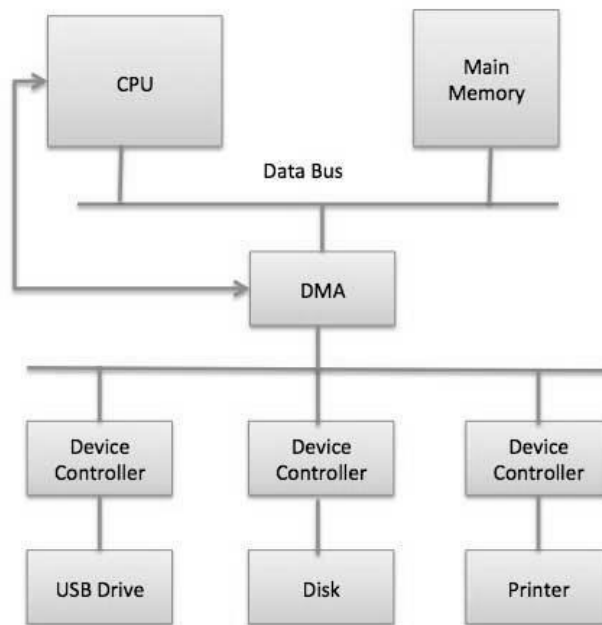


Fig:4.15: Direct Memory Access

I/O software is often organized in the following layers –

- **User Level Libraries** – This provides simple interface to the user program to perform input and output. For example, stdio is a library provided by C and C++ programming languages.
- **Kernel Level Modules** – This provides device driver to interact with the device controller and device independent I/O modules used by the device drivers.
- **Hardware** – This layer includes actual hardware and hardware controller which interact with the device drivers and makes hardware alive.

A key concept in the design of I/O software is that it should be device independent where it should be possible to write programs that can access any I/O device without having to specify the device in advance. For example, a program that reads a file as input should be able to read a file on a floppy disk, on a hard disk, or on a CD-ROM, without having to modify the program for each different device.

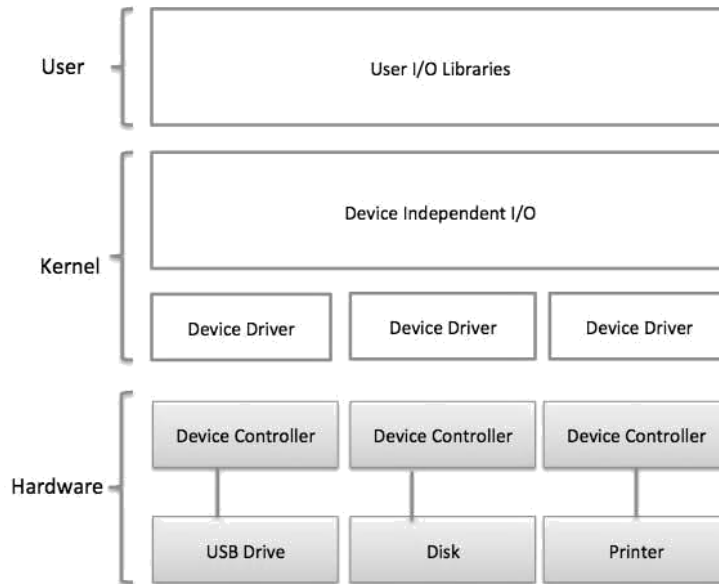


Fig:4.16: Design of I/O software

Device Drivers

Device drivers are software modules that can be plugged into an OS to handle a particular device. Operating System takes help from device drivers to handle all I/O devices. Device drivers encapsulate device-dependent code and implement a standard interface in such a way that code contains device-specific register reads/writes. Device driver, is generally written by the device's manufacturer and delivered along with the device on a CD-ROM.

A device driver performs the following jobs –

- To accept request from the device independent software above to it.
- Interact with the device controller to take and give I/O and perform required error handling
- Making sure that the request is executed successfully

How a device driver handles a request is as follows: Suppose a request comes to read a block N. If the driver is idle at the time a request arrives, it starts carrying out the request immediately. Otherwise, if the driver is already busy with some other request, it places the new request in the queue of pending requests.

Interrupt handlers

An interrupt handler, also known as an interrupt service routine or ISR, is a piece of software or more specifically a callback function in an operating system or more specifically in a device driver, whose execution is triggered by the reception of an interrupt. When the interrupt happens, the interrupt procedure does whatever it has to in order to handle the interrupt, updates data structures and wakes up process that was waiting for an interrupt to happen.

The interrupt mechanism accepts an address — a number that selects a specific interrupt handling routine/function from a small set. In most architectures, this address is an offset stored in a table called the interrupt vector table. This vector contains the memory addresses of specialized interrupt handlers.

Device-Independent I/O Software

The basic function of the device-independent software is to perform the I/O functions that are common to all devices and to provide a uniform interface to the user-level software. Though it is difficult to write completely device independent software but we can write some modules which are common among all the devices. Following is a list of functions of device-independent I/O Software –

- Uniform interfacing for device drivers
- Device naming - Mnemonic names mapped to Major and Minor device numbers
- Device protection
- Providing a device-independent block size
- Buffering because data coming off a device cannot be stored in final destination.
- Storage allocation on block devices
- Allocation and releasing dedicated devices
- Error Reporting

User-Space I/O Software

These are the libraries which provide richer and simplified interface to access the functionality of the kernel or ultimately interactive with the device drivers. Most of the user-level I/O software consists of library procedures with some exception like spooling system which is a way of dealing with dedicated I/O devices in a multiprogramming system. I/O Libraries (e.g., stdio) are in user-space to provide an interface to the OS resident device-independent I/O SW. For example putchar(), getchar(), printf() and scanf() are example of user level I/O library stdio available in C programming. RTOS generally provides large number of APIs to support diverse hardware device drivers.

TEXT / REFERENCE BOOKS

1. Jane W. S Liu, “Real Time Systems” Pearson Higher Education, 3rd Edition, 2000.
2. Raj Kamal, “Embedded Systems- Architecture, Programming and Design” Tata McGraw Hill, 2nd Edition, 2014.
3. Jean J. Labrosse, “Micro C/OS-I : The real time kernel” CMP Books, 2nd Edition, 2015.
5. Richard Barry, “Mastering the Free RTOS: Real Time Kernel”, Real Time Engineers Ltd, 1st Edition, 2016.
6. David E. Simon, “ An Embedded Software Primer”, Pearson Education Asia Publication